Studio Nx

Backend(FastAPI/Python) 02 — validation_error_handler()

2026-10-06

Backend(FastAPI/Python) 02 — `validation_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통 — 오류 응답 형식
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	01 `error_json()` · `app_error_handler()` · 03 `http_error_handler()` (404 등 기본 오류)

1. 하는 일

요청의 입력값이 API 가 정한 형식과 맞지 않을 때 FastAPI 가 내는 오류를, 칸별 문구가 담긴 422 응답으로 바꾼다.

이 함수는 유효성 검사를 하지 않는다. 검사는 이미 끝났고, 이 함수는 검사에서 떨어진 결과를 정해진 응답 모양으로 바꾼다. 더 정확한 이름은 "형식 검사 실패 응답 변환기"다.

단계	하는 곳	하는 일
① 규칙 정하기	스키마 `schemas/auth.py` 의 `SignupRequest`	"email 은 글자, 255자 이하", "nickname 은 꼭 있어야 함" 같은 규칙을 적어 둔다
② 검사하기	FastAPI · Pydantic (자동)	요청이 들어오면 ① 의 규칙과 비교한다. 틀리면 `RequestValidationError` 를 만들고 라우터를 실행하지 않는다
③ 응답 만들기	**이 함수**	② 의 오류 목록을 `{"error": {"code": "VALIDATION_ERROR", "details": {칸: 문구}}}` 로 바꾼다

예: "비밀번호 8자 이상" 규칙은 이 함수가 아니라 스키마의 `Field(min_length=8)` 에 있다. 이 함수를 지워도 검사는 그대로 일어나고, 응답만 FastAPI 기본 모양(`{"detail": [...]}`)이 된다.

유효성 검사는 세 곳에서 한다

검사	하는 곳	예	오류를 내는 방법
형식 검사	스키마 (FastAPI 가 자동으로)	칸 빠짐 · 타입 · 길이 틀림	`RequestValidationError` → **이 함수**가 422 로
규칙 검사	서비스 코드 (P1-04)	비밀번호에 대문자 · 기호가 있는지 (`check_password_rules`)	`raise AppError(VALIDATION_ERROR, 422, …)` → 01 처리기
DB 확인	서비스 코드 (P1-04)	이미 쓰는 이메일 · 닉네임	`raise AppError(EMAIL_TAKEN, 409, …)` → 01 처리기

화면(`join.js`, P1-07)도 입력하는 동안 같은 규칙을 검사하지만 사용자 편의용이다. 화면 검사는 우회할 수 있으므로 실제로 막는 것은 서버 검사다.

응답 예

가입 요청에 `nickname` 이 빠지고 비밀번호가 짧으면:

```
422  {"error": {"code": "VALIDATION_ERROR",
                "message": "Check the highlighted fields.",
                "details": {"password": "String should have at least 8 characters",
                            "nickname": "Field required"}}}
```

2. 언제 쓰나

라우터 코드가 실행되기 **전에** 생기는 오류다. 라우터는 이 오류를 직접 던지지 않는다.

순서	일어나는 일
① 요청이 들어옴	예: `POST /api/v1/auth/signup` 본문 `{"email": "a@b.com", "password": "Pw!1"}`
② FastAPI 가 입력을 스키마와 비교	라우터가 받기로 한 `SignupRequest` (`schemas/auth.py`) — 칸 이름 · 타입 · 길이
③ 맞지 않으면	`RequestValidationError` 를 만들고 **라우터를 실행하지 않음**
④ 이 처리기 실행	422 + 칸별 문구
⑤ 화면	`ApiError.details` 로 받음 — 어느 칸이 틀렸는지 알 수 있음

상황	`details`
필요한 칸이 빠짐	`{"nickname": "Field required"}`
길이 · 형식이 틀림	`{"password": "String should have at least 8 characters"}`
주소의 값이 숫자가 아님 (`/projects?page=abc`)	`{"page": "Input should be a valid integer, …"}`
본문 자체가 없음	`{"body": "Field required"}`

화면 문구는 프론트가 정한다. 가입 화면은 입력할 때 이미 규칙을 검사하고 자기 문구를 보여 준다(P1-07). 이 422 는 화면 검사를 거치지 않고 API 를 직접 부른 경우를 막는 마지막 단계이고, `details` 의 영어 문구는 개발 · 디버깅용이다.

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import 이다. 이 함수는 **`Request`** · **`RequestValidationError`** · **`JSONResponse`** 를 쓴다. `error_json` · `ErrorCode` 는 같은 파일 안에 있다.

함수

```
async def validation_error_handler(request: Request, exc: RequestValidationError) -> JSONResponse:
    details: dict[str, str] = {}
    for err in exc.errors():
        loc = err.get("loc") or ()
```

```
        field = str(loc[-1]) if loc else "body"
        details.setdefault(field, err.get("msg", "Invalid value."))
    return error_json(422, ErrorCode.VALIDATION_ERROR, "Check the highlighted fields.", details)
```

`register_exception_handlers()` 에 한 줄 추가:

```
app.add_exception_handler(RequestValidationError, validation_error_handler)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀 (예: `Request`)

식	뜻
`exc.errors()`	오류 목록. 칸마다 사전 하나: `{"loc": ("body", "nickname"), "msg": "Field required", "input": …, …}`
`err.get("loc") or ()`	`loc` 가 없거나 비었으면 빈 튜플 `()`
`loc[-1]`	마지막 값. `("body", "nickname")` → `"nickname"`
`str(...)`	목록 칸의 오류는 위치가 숫자(`("body", "tags", 0)` → `0`)라서 글자로 바꿈
`A if 조건 else B`	조건이 참이면 A, 아니면 B (JS 의 `조건 ? A : B`)
`details.setdefault(field, msg)`	`field` 가 아직 없을 때만 넣음 → **칸마다 첫 번째 문구만** 남음

loc	칸 이름
`("body", "email")`	`email`
`("query", "page")`	`page`
`("body",)` (본문 없음)	`body`
`("body", "tags", 0)`	`0`

4. 흐름

1. FastAPI 가 입력 오류 목록을 넘겨준다.
2. 오류마다 `loc` 의 마지막 값을 칸 이름으로, `msg` 를 문구로 꺼낸다.
3. 같은 칸은 첫 문구만 남긴다.
4. `error_json()` 으로 422 응답을 만든다 (01).

5. 왜 이렇게

선택	이유
FastAPI 기본 응답(`{"detail": [...]}`)을 바꿈	기본 모양은 다른 오류와 달라서 `api.js` 가 `error.code` 를 읽지 못한다. 모든 오류를 `{"error": {...}}` 하나로 맞춘다
보낸 값(`input`)은 응답에 넣지 않음	기본 응답은 사용자가 보낸 값을 그대로 돌려준다. 비밀번호 칸이 틀리면 비밀번호가 응답 · 로그에 남는다 (PRD 01 §5). 칸 이름과 문구만 보낸다
칸마다 문구 하나	화면은 칸 아래에 한 줄만 보여 준다. 사전 `{칸: 문구}` 이면 프론트가 `details.email` 로 바로 꺼낸다

선택	이유
칸 이름은 `loc` 의 마지막 값	화면의 입력 칸 이름(`email` · `nickname`)과 같아진다. 앞부분(`body` · `query`)은 화면에 필요 없다
전체 문구는 하나로 고정	칸별 내용은 `details` 에 있다. `message` 는 "표시된 칸을 확인하라"는 안내만

6. 시험 (`tests/test_errors.py` 에 3개 추가)

시험용 앱에 가입과 같은 모양의 스키마(`email` · `password` 8자 이상 · `nickname`)와 `page: int` 를 받는 주소를 만들어 확인했다.

경우	결과
이메일 형식 틀림 + `nickname` 빠짐	422 · `VALIDATION_ERROR` · `details` 의 칸이 `email`, `nickname` 두 개 · `nickname` 은 `"Field required"`
짧은 비밀번호 `Pw!1`	`details` 에 `password` · 응답 어디에도 `Pw!1` 없음
`/search?page=abc`	`details` 에 `page`
본문 없이 POST	422 · `VALIDATION_ERROR`

전체 시험 20개 통과, `ruff` 검사 통과.